Final Capstone Project Proposal

Part I — Proposal and Prototyping

STUDEND

An AI-Powered University Academic Management and Intelligent Learning Assistance System

Java Web Application with Cloud AI Integration

Prepared by	MD. Afif Zaman
Student ID	IT-23059
Course	Software Engineering and Project Management
Submitted to	Prof. Dr. Ziaur Rahman
University	Mawlana Bhashani Science and Technology University
Proposal version	1.0 — November 2026

Contents

1 Executive Summary **2**

2 Problem Statement and Motivation **2**

3 Objectives and Success Criteria **2**

4 Stakeholders and Roles **2**

5 Scope and Boundaries **3**

6 Functional Requirements **3**

7 Non-Functional Requirements **3**

8 System Architecture and Technology **4**

9 AI Academic Agent: Design and Governance **4**

10 Software Process Model **4**

11 Schedule, Milestones and Deliverables **5**

12 Data Flow Diagram (Level 0 / Context) **6**

13 Use Case Diagram **6**

14 Activity Diagram: Teacher AI Assignment **7**

15 Entity Relationship Diagram (Conceptual) **7**

16 Class Diagram (Initial Domain and Service Model) **8**

17 Object Diagram (Illustrative Snapshot) **8**

18 Prototype Interface and Demonstration **8**

19 Feasibility, Risks and Mitigations **9**

20 Testing and Evaluation Plan **9**

21 Estimated Resources and Budget **9**

22 Deliverables and GitHub Submission **9**

23 Background, Rationale and Research Questions **10**

23.1 Institutional context 10

23.2 Research and design questions 10
23.3 Project contribution 10

24 Detailed User Stories and Acceptance Scenarios 10

25 Data Model and Database Design Decisions 11
25.1 Core entities 11
25.2 Integrity constraints 11
25.3 Illustrative SQL schema fragment 11

26 Java Software Architecture and Class Responsibilities 12
26.1 Layered package organization 12
26.2 Object-oriented design 12
26.3 Request lifecycle 12
26.4 Illustrative interface contracts 12

27 AI Retrieval, Prompting and Human Oversight 13
27.1 Document processing and retrieval 13
27.2 Grounded response generation 13
27.3 Assignment generation 13
27.4 Known limitations 13

28 Security, Privacy and Responsible AI 13

29 Verification, Validation and Evaluation Methodology 14
29.1 Testing levels 14
29.2 Proposed test cases 14
29.3 AI answer evaluation 15
29.4 Completion criteria 15

30 Project Management, Dependencies and Risk Register 15

31 Deployment, Maintenance and Reproducibility 15

32 Appendix A: Proposed Screens and Navigation 16

33 Appendix B: Requirements-to-Deliverables Traceability 16

34 Conclusion 17

35 References 17

1 Executive Summary

STUDEND is a proposed role-based academic management platform for universities, integrating administration, course delivery, and AI-assisted learning. The Java web application will allow university administrators to manage students, teachers, batches, and courses; teachers to upload materials and prepare assignments; and students to access course information and an intelligent study assistant. A cloud-based language model API will support document summarization, material-grounded question answering, and teacher-reviewed assignment drafting. The project will use an iterative and incremental process and deliver an initial working prototype followed by tested enhancements.

2 Problem Statement and Motivation

Academic information is often distributed across spreadsheets, messaging applications, shared folders, and separate learning tools. This creates duplicate data entry, inconsistent access to course materials, difficulty tracking enrollment, and additional work for teachers preparing learning activities. Generic AI tools may also answer without access to the specific lecture material being studied. STUDEND proposes a unified, permission-controlled workspace with AI assistance grounded in approved course resources.

3 Objectives and Success Criteria

1. Build a Java-based web application supporting administrator, teacher, and student roles.
2. Implement persistent management of students, teachers, batches, courses, and enrollment.
3. Allow teachers to upload supported study materials and publish assignments.
4. Provide student summaries and answers linked to accessible course materials.
5. Allow teachers to generate editable assignment drafts and publish only after approval.
6. Demonstrate at least five interacting Java domain classes and database persistence.
7. Deliver UML/DFD models, test evidence, source code, and GitHub documentation.

Illustrative acceptance criteria. A user can sign in with the correct role; an admin can create and view a course with assigned teacher and batch; a teacher can upload a supported document and review an AI-generated assignment draft; a student enrolled in the course can request a summary and receive a source-grounded answer; unauthorized users cannot access another course's private materials. AI quality will be evaluated using a small, manually checked question set.

4 Stakeholders and Roles

Stakeholder	Primary needs
University administrator	Member management, batch and course assignment, academic overview
Teacher	Course materials, assignment creation and approval, student engagement
Student	Assigned courses, accessible resources, summaries and study Q&A
System maintainer	Security, backups, error monitoring, configuration and API budgets

5 Scope and Boundaries

Initial prototype (minimum demonstrable product). Login and role routing; administrator management of student, teacher, batch and course records; basic teacher and student dashboards; course material upload; student AI summary/Q&A for a selected material; AI-generated assignment draft requiring teacher approval; MySQL persistence. **Planned full system.** Attendance, grading, Excel import, academic timetables, course archives, advanced search, citation-aware retrieval across many documents, richer assessment workflows, analytics, and multi-university onboarding. **Out of scope for initial prototype.** Automated final grading, AI publication without teacher review, fully autonomous agent actions, and deployment to a production university without institutional approval.

6 Functional Requirements

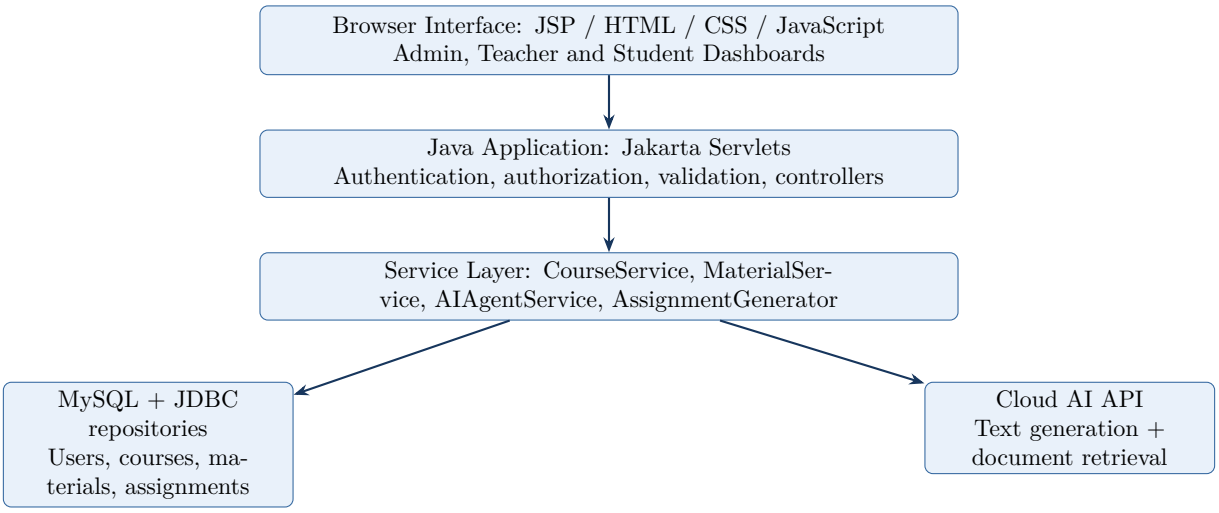
ID	Requirement
FR-01	Authenticate users and enforce administrator, teacher, and student permissions.
FR-02	Create, update, list, and deactivate student and teacher memberships.
FR-03	Create batches and associate students with one or more batches.
FR-04	Create courses and assign one teacher and one batch per course offering.
FR-05	Display course lists appropriate to each authenticated user's role.
FR-06	Allow authorized teachers to upload and manage supported course materials.
FR-07	Allow enrolled students to request summaries of accessible course materials.
FR-08	Answer study questions using retrieved excerpts from authorized materials and provide source references when available.
FR-09	Allow teachers to specify topic, question type, difficulty, marks and instructions for AI assignment generation.
FR-10	Store generated assignments as drafts until the teacher reviews, edits and explicitly publishes them.
FR-11	Record AI requests, outcome status, and usage metadata subject to privacy and retention settings.
FR-12	Reject unauthorized resource access and display useful errors when the cloud API is unavailable.

7 Non-Functional Requirements

- **Security:** salted password hashes, authenticated sessions, role-based authorization on every server endpoint, protected API credentials, validated uploads, parameterized SQL.
- **Privacy:** minimize personal information sent to the cloud AI provider; disclose third-party processing; respect institutional permission and retention requirements.
- **Reliability:** database transactions for multi-step changes; meaningful error handling and backup/recovery procedures.
- **Usability:** responsive, accessible navigation and clear status labels for AI-generated drafts.
- **Performance:** responsive basic CRUD operations on the demonstration dataset; AI calls use loading indicators, timeouts, and reasonable rate limits.

- **Maintainability:** Maven project structure, layered services and repositories, consistent naming, and unit/integration tests.
- **Cost control:** per-user request limits, usage monitoring, and configurable model selection; actual API fees depend on provider and usage.

8 System Architecture and Technology



The application will be built using Java, Jakarta Servlet API, JSP, JDBC, MySQL, Maven and Apache Tomcat. The AI integration can use the official OpenAI Java SDK and Responses API; hosted file search is a candidate retrieval option. Alternatively, the Java service can extract document text and retrieve approved excerpts locally before sending only those excerpts to the API. The implementation decision will be validated during the AI integration milestone.

9 AI Academic Agent: Design and Governance

Student workflow. After confirming course enrollment, the service identifies the selected document or permitted course collection, retrieves relevant content, sends a constrained prompt to the cloud API, and returns a summary or answer with document attribution. When evidence is insufficient, the assistant must state that the available material does not support the answer. **Teacher workflow.**

A teacher selects a course, topic/material, question format, number of questions, difficulty and marks. The AI drafts questions and an optional marking rubric. The draft is stored separately from published assignments. The teacher can edit, discard or approve it; publishing requires an explicit teacher action. **Safeguards.** Apply authorization before retrieval; never allow model output to override server permissions; do not send passwords or unnecessary student identifiers; label AI-generated content; treat uploaded documents as untrusted text; provide rate limiting, auditing and teacher oversight. The system must not expose unpublished assignment answers to students.

10 Software Process Model

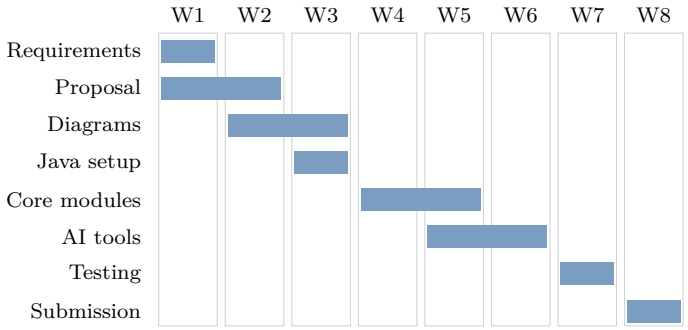
Selected model: Iterative and Incremental Development. The first increment establishes the database and authenticated roles; the second implements academic administration; the third

adds material management and AI assistance; the fourth focuses on testing, refinement, and documentation. This model fits a capstone because each increment produces demonstrable functionality while allowing feedback and controlled revisions.

11 Schedule, Milestones and Deliverables

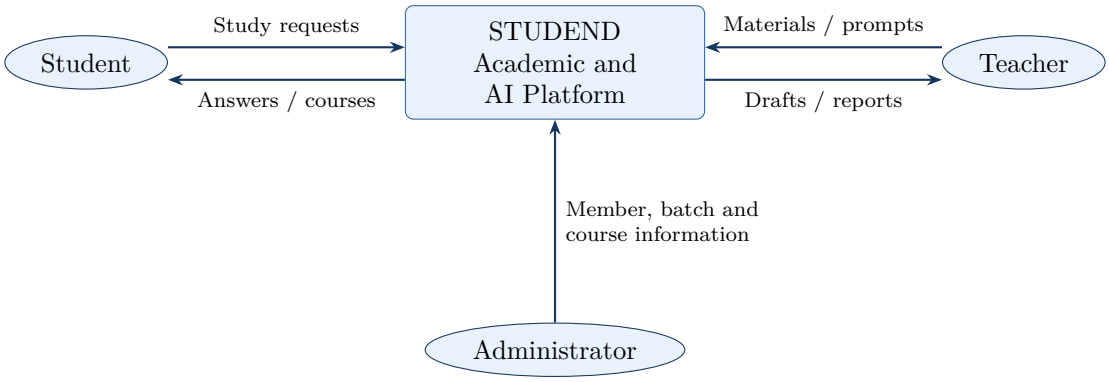
The following eight-week plan is a *proposed* schedule, to be aligned with the actual course calendar; it does not imply work was completed before the assignment deadline.

Work package	Scheduled weeks
Requirements and feasibility	W1
Proposal and process plan	W1–W2
Diagrams and schema	W2–W3
Java/Maven foundation	W3
Core academic modules	W4–W5
AI material and assignment tools	W5–W6
Testing and correction	W7
Documentation and submission	W8

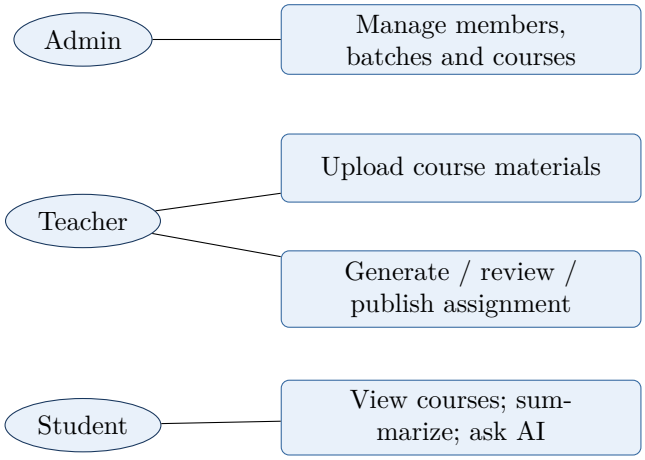


Milestone	Target	Verifiable deliverable
M1	End W2	Proposal, requirements baseline, process model
M2	End W3	Six diagrams, relational schema, Maven skeleton
M3	End W5	Role-based CRUD prototype with five or more Java classes
M4	End W6	Material-grounded Q&A and teacher assignment draft demo
M5	End W8	Test report, screenshots, README, GitHub repository

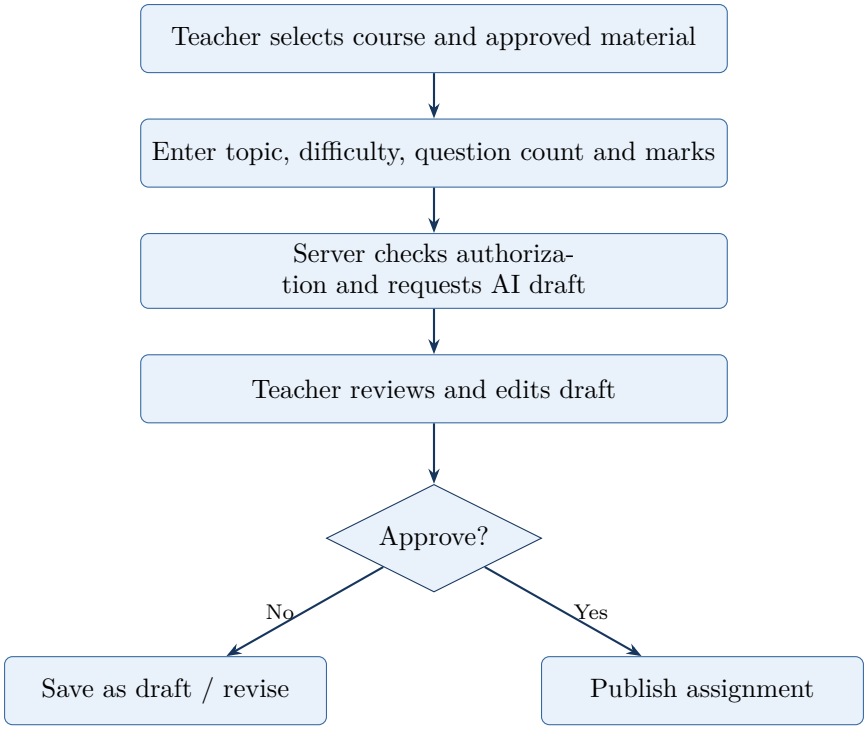
12 Data Flow Diagram (Level 0 / Context)



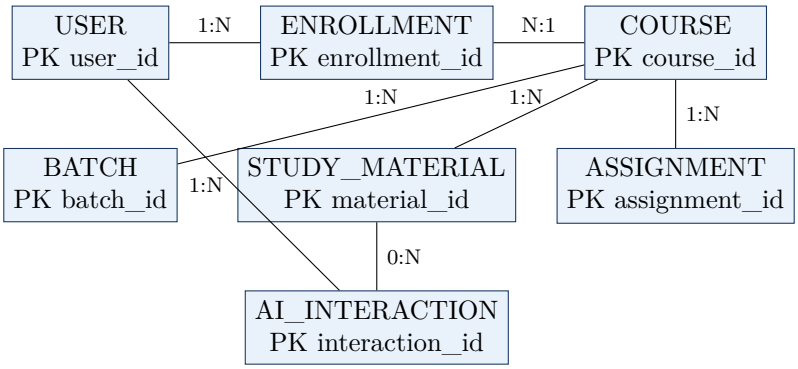
13 Use Case Diagram



14 Activity Diagram: Teacher AI Assignment

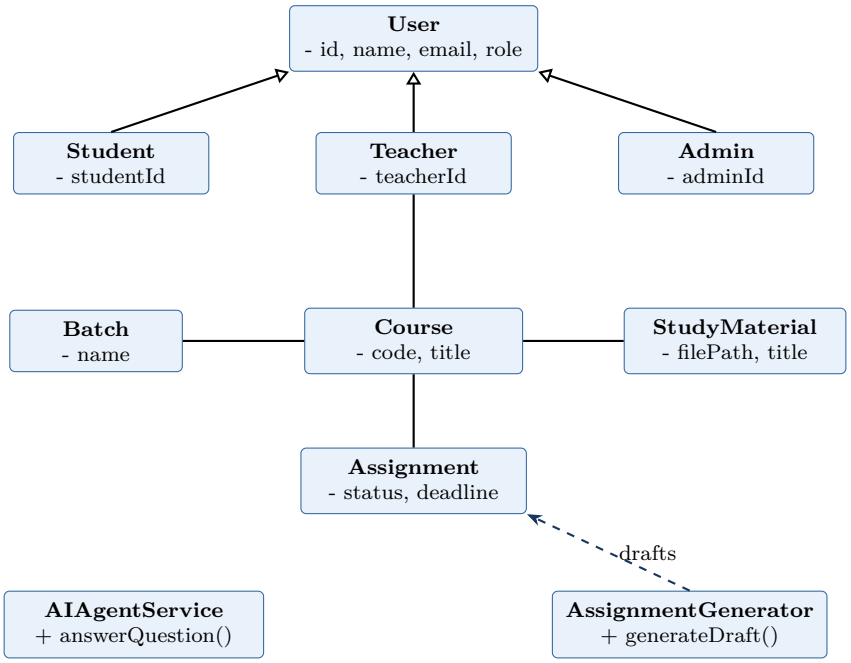


15 Entity Relationship Diagram (Conceptual)



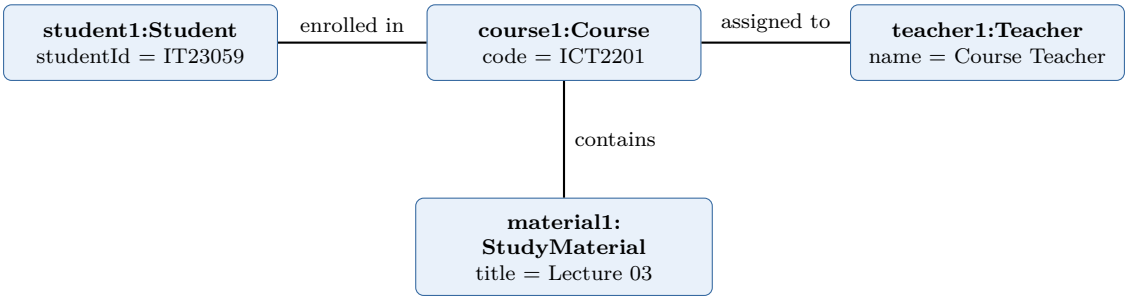
The logical schema will additionally enforce teacher-course assignments, university tenancy and batch membership. The diagram is simplified for proposal readability; cardinalities and keys will be refined during database design.

16 Class Diagram (Initial Domain and Service Model)



The prototype will implement at least `User`, `Student`, `Teacher`, `Batch`, `Course`, `StudyMaterial`, and `Assignment`, supported by servlet controllers and JDBC repositories. The inheritance and associations shown are conceptual and will be checked against the implementation.

17 Object Diagram (Illustrative Snapshot)



These objects are sample instances for the diagram and do not represent verified production records.

18 Prototype Interface and Demonstration

Page	Demonstrated interactions
Login / role routing	Authenticate and display role-specific dashboard
Admin dashboard	Create/list students, teachers, batches and courses
Teacher course page	Upload material; request AI assignment draft; edit and publish
Student course page	View assigned material; request summary; ask a question
AI response view	Display answer, source reference, warning when insufficient evidence

19 Feasibility, Risks and Mitigations

Risk or constraint	Mitigation
API cost and connectivity	Request quotas, model configuration, retry policy, clear fallback UI
Hallucinated AI content	Material-grounded retrieval, citations, uncertainty disclosure, human review
Data privacy	Authorization checks, data minimization, institution-approved uploads, retention controls
Unauthorized course access	Enforce membership checks server-side for every file and AI request
Malicious or oversized files	Allowed file types, size limits, safe storage, content validation
Time and scope overrun	Incremental milestones; prioritize five-class working prototype
Database inconsistencies	Foreign keys, constraints, transactions and test fixtures

20 Testing and Evaluation Plan

Unit tests will validate entities and business rules; integration tests will verify JDBC persistence, servlet authorization, and course assignment; functional tests will exercise each role's main workflow; negative tests will attempt unauthorized material access. For AI, a curated set of questions will be compared with lecture content, checking factual grounding, attribution, and appropriate abstention. Teacher-generated assignments will be evaluated for topic relevance, clarity, marks allocation, and whether approval is required before publication. No quantitative accuracy claims will be made until testing is completed.

21 Estimated Resources and Budget

Development tools (JDK, Maven, MySQL community tools, Git) can be used without software licensing fees under their respective terms. The prototype requires a development computer, a Tomcat server, database storage, network access for AI calls, and an API account. Cloud API and hosting charges are variable and will be measured rather than assumed; a fixed request quota will be used during the demonstration.

22 Deliverables and GitHub Submission

1. Final LaTeX proposal and compiled PDF.
2. Six required diagrams and editable diagram sources.
3. Maven Java source code, JSP/CSS/JS assets, and database setup scripts.
4. Screenshots of the prototype and test results.
5. README with architecture, prerequisites, build/run instructions, feature status, and limitations.
6. Public GitHub repository URL submitted through the assignment portal.

23 Background, Rationale and Research Questions

23.1 Institutional context

A typical university course involves several interconnected records: students belong to batches, teachers are assigned to course offerings, and learning resources and assessments must be available to the correct participants. When these records are maintained separately, staff must reconcile conflicting versions and students may struggle to locate authoritative material. STUDEND treats the course offering as the central unit connecting people, content, and learning activities.

23.2 Research and design questions

The project will investigate three practical questions: (1) Can a role-aware Java web application reduce repeated administrative operations in a representative course-management workflow? (2) Can retrieval of approved study materials improve the relevance and traceability of AI-generated study answers? (3) Can a human-in-the-loop workflow make AI-generated assignment drafts useful without giving the AI authority to publish assessments?

23.3 Project contribution

The proposed contribution is an integrated prototype rather than a new foundation model. Its distinctive feature is the combination of academic permissions, course-specific retrieval, and teacher-controlled AI drafting within a conventional Java enterprise-style architecture.

24 Detailed User Stories and Acceptance Scenarios

ID	User story	Acceptance evidence
US-01	As an administrator, I want to register a student so the student can access assigned courses.	Valid data creates a membership; duplicate active email is rejected with an explanation.
US-02	As an administrator, I want to assign a teacher and batch to a course.	The course appears in the assigned teacher and enrolled students' dashboards.
US-03	As a teacher, I want to upload lecture notes for my course.	A permitted file is stored, indexed or extracted, and visible only to authorized users.
US-04	As a student, I want to summarize a lecture.	A concise summary is returned and identifies the source material used.
US-05	As a student, I want to ask a question about lecture content.	The answer uses retrieved material and admits when evidence is missing.
US-06	As a teacher, I want AI to propose assignment questions.	The draft reflects specified topic, type, difficulty and marks, and remains unpublished.
US-07	As a teacher, I want to edit and publish a draft.	Only an authorized teacher can publish, and publication is logged.
US-08	As a student, I want my course resources protected.	Requests for a course in which I am not enrolled are denied.

Scenario A: Student learning assistance. A student logs in, opens an enrolled course, selects an uploaded lecture, and requests an explanation of a topic. The application checks enrollment

before retrieving text. The response shows a useful explanation, identifies the material consulted, and marks any unsupported claim as uncertain. If no content is available, the interface explains the limitation instead of fabricating a source. **Scenario B: Teacher assignment drafting.** A teacher opens an assigned course, chooses a source lecture, requests five short-answer questions with a specified total mark, and reviews the proposed questions and rubric. The draft is saved with a *draft* status. Students cannot see it until the teacher explicitly publishes it.

25 Data Model and Database Design Decisions

25.1 Core entities

The relational schema will separate identity, institutional membership, course enrollment, teaching resources, and AI activities. This separation reduces duplication and makes authorization checks explicit.

Entity	Important attributes and purpose
User	user_id (PK), name, email (unique), password_hash, account_status; global login identity.
University	university_id (PK), name, active_status; owning institution.
Membership	membership_id (PK), user_id (FK), university_id (FK), role, status; institutional access.
Batch	batch_id (PK), university_id (FK), name; student grouping.
BatchMember	batch_id (FK), membership_id (FK), joined_at; many-to-many membership.
Course	course_id (PK), university_id (FK), code, title, teacher_membership_id (FK), batch_id (FK), status.
StudyMaterial	material_id (PK), course_id (FK), uploader_id (FK), title, storage_path, file_type, status.
MaterialChunk	chunk_id (PK), material_id (FK), chunk_index, extracted_text, source_locator; retrievable passages.
Assignment	assignment_id (PK), course_id (FK), creator_id (FK), title, status, published_at.
Question	question_id (PK), assignment_id (FK), prompt, marks, type, sequence.
AIInteraction	interaction_id (PK), requester_id (FK), course_id (FK), purpose, created_at, outcome; minimal audit metadata.

25.2 Integrity constraints

Foreign keys will prevent references to nonexistent courses or users. Enrollment uniqueness will be enforced for each batch-member pair. Every repository query for course content will be scoped to the authenticated university and course. Soft deactivation is preferable for memberships and published academic records; destructive deletion requires explicit handling of dependent rows and should not be the default.

25.3 Illustrative SQL schema fragment

```
CREATE TABLE study_material (
  material_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  course_id BIGINT NOT NULL,
```

```
  uploader_id BIGINT NOT NULL,
  title VARCHAR(255) NOT NULL,
  storage_path VARCHAR(500) NOT NULL,
  processing_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (course_id) REFERENCES course(course_id)
);
```

This fragment illustrates the intended design and is not a claim that the complete database schema has already been executed.

26 Java Software Architecture and Class Responsibilities

26.1 Layered package organization

```
src/main/java/com/studend/
  model/       User, Student, Teacher, Batch, Course, Assignment
  repository/  UserRepository, CourseRepository, MaterialRepository
  service/     AuthService, CourseService, MaterialService
               AIAgentService, AssignmentGenerator
  servlet/     LoginServlet, CourseServlet, AIChatServlet
               AssignmentServlet, MaterialUploadServlet
  security/    AuthorizationFilter, SessionUtil
src/main/webapp/WEB-INF/views/
  login.jsp, admin-dashboard.jsp, teacher-dashboard.jsp
  student-dashboard.jsp, ai-chat.jsp, assignment-editor.jsp
```

26.2 Object-oriented design

The prototype will include more than five Java domain classes, including `User`, `Student`, `Teacher`, `Batch`, `Course`, `StudyMaterial`, and `Assignment`. The application will use encapsulation for entity state, interfaces for repository abstractions, and composition between services and repositories. Student and Teacher may specialize User at the conceptual level, while persistence may use a single User table plus role-based memberships. The final class diagram will document whichever mapping is actually implemented.

26.3 Request lifecycle

For a normal course-management request, the browser submits an HTTP request to a servlet. The servlet validates input and checks session authorization, delegates to a service, and returns a JSP view or structured response. The service calls a repository using parameterized JDBC statements. For AI requests, the service first checks course access, then calls the retrieval component and cloud model client. The cloud API credential is loaded from server configuration rather than embedded in a JSP or GitHub repository.

26.4 Illustrative interface contracts

```
interface MaterialRepository {
    List<StudyMaterial> findAccessibleByCourse(long courseId);
    Optional<StudyMaterial> findById(long materialId);
}
interface AcademicAIService {
```

```
    StudyAnswer answerQuestion(long userId, long courseId,
                               String question);
    AssignmentDraft generateDraft(long teacherId,
                                  AssignmentRequest request);
}
```

These interfaces are proposed contracts; implementations and method signatures may evolve during prototyping.

27 AI Retrieval, Prompting and Human Oversight

27.1 Document processing and retrieval

The initial system will accept a controlled set of text-based PDF and plain-text files. Uploaded files are validated by type and size, stored outside the public web directory, and associated with a course. Extracted text is divided into passages with document identifiers and page or section locators when available. For a student question, the retrieval layer selects relevant passages from authorized materials only. A later increment may use embeddings or a managed file-search service for semantic retrieval.

27.2 Grounded response generation

The AI request will contain the student's question, retrieved excerpts, and instructions to answer only from the provided material. The output will distinguish evidence-backed explanations from uncertainty and include document attribution where supported. Prompt instructions do not replace backend access controls; the server must filter sources before sending any text to the model.

27.3 Assignment generation

The teacher supplies the course, topic, learning objective, format, difficulty, question count, marks and optional source materials. The model produces a structured draft. The Java service validates the requested number of questions, total marks and response format, then saves it as a draft. A teacher can revise, discard or publish it. The AI must not directly update publication status.

27.4 Known limitations

Cloud-generated text can be inaccurate, incomplete, biased or inconsistent. Scanned PDFs may require OCR that is outside the initial prototype. Citations may refer to retrieved sections without proving every sentence. API latency, rate limits, outages and variable fees may affect the demonstration. Evaluation and human review are therefore part of the design, not optional additions.

28 Security, Privacy and Responsible AI

Risk area	Planned control
Authentication	Store salted password hashes; rotate session identifiers at login; expire inactive sessions.
Authorization	Check university membership, course enrollment and teacher ownership in every relevant service method.

Risk area	Planned control
File uploads	Restrict allowed formats, validate size and MIME type, sanitize display names, store outside public paths.
SQL injection	Use prepared statements and strict server-side validation.
Cloud API secret	Keep the key in server environment variables or a protected secret store; never commit it to Git.
Data minimization	Send only necessary excerpts and question context; avoid student IDs and private grades in prompts.
Prompt injection	Treat retrieved documents as data, never as instructions authorizing actions or data access.
AI transparency	Label generated drafts, show uncertainty, and require teacher approval before publication.
Availability	Use request timeouts, clear fallback errors, basic rate limiting and usage quotas.
Auditability	Record actor, action, resource and outcome without unnecessarily storing sensitive prompt content.

Before any real institutional deployment, the university must approve its data processing practices, retention policy, cloud provider arrangement and student-facing notices. Prototype demonstrations should use synthetic or explicitly permitted data.

29 Verification, Validation and Evaluation Methodology

29.1 Testing levels

Unit tests will cover validators, role checks, course assignment rules, and assignment draft state transitions. **Integration tests** will exercise JDBC repositories against a test database and verify that unauthorized course queries return no records. **System tests** will follow end-to-end scenarios through the browser. **User acceptance tests** will ask representative student and teacher users to complete tasks using a checklist.

29.2 Proposed test cases

Test	Action	Expected result
T-01	Sign in with valid student credentials.	Student dashboard opens.
T-02	Request admin endpoint as a student.	Access denied; no state change.
T-03	Create a course with teacher and batch.	Record persists; correct users see it.
T-04	Upload an unsupported file type.	Upload rejected safely.
T-05	Ask about a document in another course.	No private content is returned.
T-06	Ask a question unsupported by selected notes.	Response states insufficient evidence.
T-07	Generate an assignment draft.	Draft is editable and not student-visible.
T-08	Publish a draft as an unauthorized user.	Request denied.
T-09	Simulate cloud API timeout.	Clear error; application remains usable.

29.3 AI answer evaluation

A small benchmark will be created from permitted sample lecture notes, with factual questions and a teacher-reviewed answer key. Responses will be scored for factual correctness, relevance to the source, attribution quality, and appropriate abstention when evidence is absent. Assignment drafts will be checked for alignment with the requested topic, clarity, difficulty, mark totals, and editing effort. Results will be reported as measured observations rather than guaranteed accuracy claims.

29.4 Completion criteria

The prototype will be considered demonstrable when at least five domain classes participate in working features, key records persist in MySQL, role checks prevent unauthorized access, a student can receive one grounded AI response, and a teacher can review and publish an AI-generated assignment draft. Any unmet criteria will be documented honestly.

30 Project Management, Dependencies and Risk Register

Risk	Likelihood	Impact	Response
Scope expansion	High	High	Freeze the minimum prototype; move extras to backlog.
Cloud API outage	Medium	Medium	Show fallback error and use recorded, clearly labeled demo examples for UI tests.
AI inaccuracy	High	High	Source retrieval, explicit uncertainty and teacher review.
Database integration	Medium	High	Implement schema and repository tests early.
Document parsing	Medium	Medium	Begin with text PDFs; exclude scans initially.
Schedule slippage	Medium	High	Deliver working increments and prioritize critical paths.
Sensitive data exposure	Medium	High	Test authorization and minimize cloud-transmitted data.
Cost overrun	Medium	Medium	Set request limits, monitor tokens and define a spending ceiling.

Critical dependencies. Course and membership records must exist before secure material retrieval can be tested. Document extraction must work before grounded AI answers can be evaluated. Draft assignment storage must exist before teacher publication workflows can be demonstrated. These dependencies justify the incremental development order. **Change management.** New requirements will be recorded in a backlog, assigned a priority, and evaluated against milestone deadlines. Features that do not support minimum acceptance criteria will be deferred rather than destabilizing the prototype.

31 Deployment, Maintenance and Reproducibility

The demonstrator will run locally using JDK, Maven, Tomcat 11 and MySQL. The GitHub repository will contain a setup guide, example environment configuration without secrets, database creation scripts, test data instructions, and screenshots. A clean installation should be reproducible by cloning the repository, configuring the database and cloud API key, building with Maven, deploying

the WAR to Tomcat, and running documented smoke tests. **Operations plan.** The design will include database backup procedures, basic error logging, configuration for AI rate limits, and a manual method to disable AI calls without disabling core course management. A later deployment can add HTTPS termination, automated CI, monitoring, and institutional single sign-on. **Deliverable traceability.** The proposal describes intended features; the repository README will distinguish implemented, partially implemented and planned functionality. Diagram revisions will be committed alongside major architecture changes. Screenshots will be taken from the actual working application, not generated mockups presented as completed software.

32 Appendix A: Proposed Screens and Navigation

Screen	Main elements
Login	Email, password, role-aware redirect and error messages.
Admin dashboard	Overview counts and links to student, teacher, batch and course management.
Manage students	Searchable table, add/edit/deactivate forms and validation messages.
Manage courses	Course code, title, assigned teacher, batch, status and actions.
Teacher course page	Materials list, upload control, assignments list and AI draft generator.
Student course page	Published resources, assignments and AI study assistant entry point.
AI study assistant	Material selector, question input, answer panel, source labels and limitations notice.
Assignment editor	Generated questions, marks, edit controls, save draft and publish confirmation.

33 Appendix B: Requirements-to-Deliverables Traceability

Requirement group	Implementation artifact	Verification artifact
FR-01–05	Auth, memberships, batch and course services	Role/access and CRUD test cases
FR-06	Upload servlet, MaterialService, StudyMaterial	Upload and course access tests
FR-07–08	AIChatServlet, retrieval, AIAgentService	Grounded answer benchmark and denial tests
FR-09–10	AssignmentGenerator, editor, Assignment	Draft, edit and publish workflow tests
FR-11–12	AIInteraction logging and exception handling	Audit record and timeout tests
Process model	Iteration plan, milestones, Gantt chart	Weekly deliverable checklist
Diagrams	DFD, Use Case, Activity, ER, Class, Object	Diagram-to-code review
Submission	GitHub repository and proposal PDF	README, screenshots and setup verification

34 Conclusion

STUDEND combines conventional academic administration with a course-aware AI assistant to improve access to learning materials and reduce repetitive assignment preparation. A Java Servlet/JSP architecture, MySQL persistence, explicit authorization, and teacher-controlled publication make the proposal feasible as an incremental capstone. The first prototype will demonstrate a coherent subset of the design; later increments will extend functionality after validation.

35 References

1. OpenAI, *OpenAI Java API Library*, https://developers.openai.com/api/reference/java.
2. OpenAI, *File Search Guide*, https://developers.openai.com/api/docs/guides/tools-file-search.
3. OpenAI, *Responses API*, https://developers.openai.com/api/reference/java/resources/responses/methods/create.
4. Oracle, *Java Platform Documentation*, https://docs.oracle.com/en/java/.
5. Apache Software Foundation, *Apache Tomcat Documentation*, https://tomcat.apache.org/.